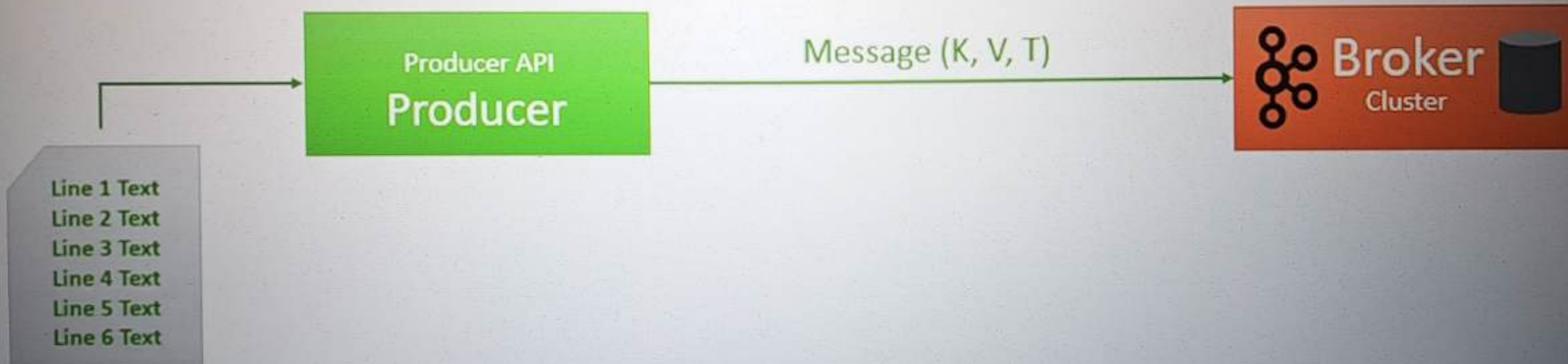




14-kafka-producer Python ▾

File Edit View Run Help Last edit was 18 minutes ago Provide feedback

▶ Run all

● my cluster ▾

Share

Publish

Cmd 1

Python ▶ ▾ - x

```
class KafkaProducer():  
    def __init__(self):  
        self.base_data_dir = "/FileStore/data_spark_streaming_scholarnest"  
        self.Bootstrap_SERVER = "pkc-6ojv2.us-west4.gcp.confluent.cloud:9092"  
        self.JAAS_MODULE = "org.apache.kafka.common.security.plain.PlainLoginModule"  
        self.CLUSTER_API_KEY = "TQ7F3730G6J4TT4G"  
        self.CLUSTER_API_SECRET = "ftb0mW0YcCT6oAG9GSIjyUxHyu7mbUPD10G0cSymt0t6FKJ1J11VDb0RVqKoG5H4"  
  
    def getSchema(self):  
        return """InvoiceNumber string, CreatedTime bigint, StoreID string, PosID string, CashierID string,  
        CustomerType string, CustomerCardNo string, TotalAmount double, NumberOfItems bigint,  
        PaymentMethod string, TaxableAmount double, CGST double, SGST double, CESS double,  
        DeliveryType string,  
        DeliveryAddress struct<AddressLine string, City string, ContactNumber string, PinCode string,  
        State string>,  
        InvoiceLineItems array<struct<ItemCode string, ItemDescription string,  
        ItemPrice double, ItemQty bigint, TotalValue double>>  
        """"  
  
    def readInvoices(self, condition):  
        from pyspark.sql.functions import expr  
        return (spark.readStream  
                .format("json")  
                .schema(self.getSchema())  
                .load(f"{self.base_data_dir}/data/invoices")  
                .where(condition)  
                )  
  
    def getKafkaMessage(self, df, key):  
        return df.selectExpr(f"{key} as key", "to_json(struct(*)) as value")  
  
    def sendToKafka(self, kafka_df):
```



Cmd 1

Python

```
class KafkaProducer():
    def __init__(self):
        self.base_data_dir = "/FileStore/data_spark_streaming_scholarnest"
        self.Bootstrap_SERVER = "pkc-6ojv2.us-west4.gcp.confluent.cloud:9092"
        self.JAAS_MODULE = "org.apache.kafka.common.security.plain.PlainLoginModule"
        self.CLUSTER_API_KEY = "TQ7F3730G6J4TT4G"
        self.CLUSTER_API_SECRET = "ftb0mWOYcCT6oAG9GSIjyUxHYuJmbUPD10G0c5ymt0t6FKJ1j1IVDbDRVqKoG5H4"

    def getSchema(self):
        return """InvoiceNumber string, CreatedTime bigint, StoreID string, PosID string, CashierID string,
        CustomerType string, CustomerCardNo string, TotalAmount double, NumberOfItems bigint,
        PaymentMethod string, TaxableAmount double, CGST double, SGST double, CESS double,
        DeliveryType string,
        DeliveryAddress struct<Addressline string, City string, ContactNumber string, PinCode string,
        State string>,
        InvoiceLineItems array<struct<ItemCode string, ItemDescription string,
        ItemPrice double, ItemQty bigint, TotalValue double>>
        """

    def readInvoices(self, condition):
        from pyspark.sql.functions import expr
        return (spark.readStream
                .format("json")
                .schema(self.getSchema())
                .load(f"{self.base_data_dir}/data/invoices")
                .where(condition)
                )

    def getKafkaMessage(self, df, key):
        return df.selectExpr(f"{key} as key", "to_json(struct(*)) as value")

    def sendToKafka(self, kafka df):
```



14-kafka-producer Python

File Edit View Run Help Last edit was 18 minutes ago Provide feedback

Run all

my cluster

Share

Publish

```
.option("kafka.sasl.mechanism", "PLAIN")
.option("kafka.sasl.jaas.config", f"{self.JAAS_MODULE} required username='{self.CLUSTER_API_KEY}' password='{self.CLUSTER_API_SECRET}';")
.option("topic", "invoices")
.option("checkpointLocation", f"{self.base_data_dir}/checkpoint/kafka_producer")
.outputMode("append")
.start()
)
```

```
def process(self, condition):
    print(f"Starting Kafka Producer Stream...", end='')
    invoices_df = self.readInvoices(condition)
    kafka_df = self.getKafkaMessage(invoices_df)
    sQuery = self.sendToKafka(kafka_df)
    print("Done\n")
    return sQuery
```

Command took 0.07 seconds -- by prashantkumarpandey@gmail.com at 10/26/2023, 8:06:09 PM on my cluster

Shift+Enter to run

Shift+Ctrl+Enter to run selected text



14-kafka-producer Python

File Edit View Run Help Last edit was 2 minutes ago Provide feedback

Run all

my cluster

Show

Publish

```
State string>,  
InvoiceLineItems array<struct<ItemCode string, ItemDescription string,  
ItemPrice double, ItemQty bigint, TotalValue double>>  
---
```

```
def readInvoices(self, condition):  
    from pyspark.sql.functions import expr  
    return (spark.readStream  
            .format("json")  
            .schema(self.getSchema())  
            .load(f"{self.base_data_dir}/data/invoices")  
            .where(condition)  
            )  
  
def getKafkaMessage(self, df, key):  
    return df.selectExpr(f"{key} as key", "to_json(struct(*)) as value")  
  
def sendToKafka(self, kafka_df):  
    return (kafka_df.writeStream  
            .format("kafka")  
            .option("kafka.bootstrap.servers", self.BOOTSTRAP_SERVER)  
            .option("kafka.security.protocol", "SASL_SSL")  
            .option("kafka.sasl.mechanism", "PLAIN")  
            .option("kafka.sasl.jaas.config", f"{self.JAAS_MODULE} required username='{self.CLUSTER_API_KEY}' password='{self.CLUSTER_API_PASSWORD}'")  
            .option("topic", "invoices")  
            .option("checkpointLocation", f"{self.base_data_dir}/checkpoint/kafka_producer")  
            .outputMode("append")  
            .start()  
            )  
  
def process(self, condition):  
    print(f"Starting Kafka Producer Stream...", end='')  
    invoices_df = self.readInvoices(condition)  
    kafka_df = self.getKafkaMessage(invoices_df)  
    sQuery = self.sendToKafka(kafka_df)
```



```
from pyspark.sql.functions import expr
return (spark.readStream
        .format("json")
        .schema(self.getSchema())
        .load(f"{self.base_data_dir}/data/invoices")
        .where(condition)
)

def getKafkaMessage(self, df, key):
    return df.selectExpr(f"{key} as key", "to_json(struct(*)) as value")

def sendToKafka(self, kafka_df):
    return (kafka_df.writeStream
            .format("kafka")
            .option("kafka.bootstrap.servers", self.BOOTSTRAP_SERVER)
            .option("kafka.security.protocol", "SASL_SSL")
            .option("kafka.sasl.mechanism", "PLAIN")
            .option("kafka.sasl.jaas.config", f"{self.JAAS_MODULE} required username='{self.CLUSTER_API_KEY}' password='{self.CLUSTER_API_SECRET}';")
            .option("topic", "invoices")
            .option("checkpointlocation", f"{self.base_data_dir}/checkpoint/kafka_producer")
            .outputMode("append")
            .start()
    )

def process(self, condition):
    print(f"Starting Kafka Producer Stream...", end='')
    invoices_df = self.readInvoices(condition)
    kafka_df = self.getKafkaMessage(invoices_df)
    sQuery = self.sendToKafka(kafka_df)
    print("Done\n")
    return sQuery
```



14-kafka-producer Python ▾

File Edit View Run Help [Last edit was 9 minutes ago](#) [Provide feedback](#)

▶ Run all

● my cluster ▾

Share

Publish

```
from pyspark.sql.functions import expr
return (spark.readStream
        .format("json")
        .schema(self.getSchema())
        .load(f"{self.base_data_dir}/data/invoices")
        .where(condition)
        )

def getKafkaMessage(self, df, key):
    return df.selectExpr(f"{key} as key", "to_json(struct(*)) as value")

def sendToKafka(self, kafka_df):
    return (kafka_df.writeStream
            .format("kafka")
            .option("kafka.bootstrap.servers", self.BOOTSTRAP_SERVER)
            .option("kafka.security.protocol", "SASL_SSL")
            .option("kafka.sasl.mechanism", "PLAIN")
            .option("kafka.sasl.jaas.config", f"{self.JAAS_MODULE} required username='{self.CLUSTER_API_KEY}' password='{self.CLUSTER_API_SECRET}';")
            .option("topic", "invoices")
            .option("checkpointlocation", f"{self.base_data_dir}/checkpoint/kafka_producer")
            .outputMode("append")
            .start()
            )

def process(self, condition):
    print(f"Starting Kafka Producer Stream...", end='')
    invoices_df = self.readInvoices(condition)
    kafka_df = self.getKafkaMessage(invoices_df)
    sQuery = self.sendToKafka(kafka_df)
    print("Done\n")
    return sQuery
```

Command took 0.07 seconds -- by prashantkumarpandey@gmail.com at 10/28/2023, 8:06:09 PM on my cluster




```
class KafkaProducerTestSuite():  
    def __init__(self):  
        self.base_data_dir = "/FileStore/data_spark_streaming_scholarnest"  
  
    def cleanTests(self):  
        print(f"Starting Cleanup...", end='')  
        dbutils.fs.rm(f"{self.base_data_dir}/checkpoint/kafka_producer", True)  
        dbutils.fs.rm(f"{self.base_data_dir}/data/invoices", True)  
        dbutils.fs.mkdirs(f"{self.base_data_dir}/data/invoices")  
        print("Done")  
  
    def ingestData(self, itr):  
        print(f"\tStarting Ingestion...", end='')  
        dbutils.fs.cp(f"{self.base_data_dir}/datasets/invoices/invoices_{itr}.json", f"{self.base_data_dir}/data/invoices/")  
        print("Done")
```


15-kafka-producer-test-suit Python ▾

File Edit View Run Help [Last edit was 3 minutes ago](#) [Provide feedback](#)

▶ Run all

● Connect ▾

```
print(f"\tStarting Ingestion...", end='')
dbutils.fs.cp(f"{self.base_data_dir}/datasets/invoices/invoices_{itr}.json", f"{self.base_data_dir}/data/invoices/")
print("Done")
```

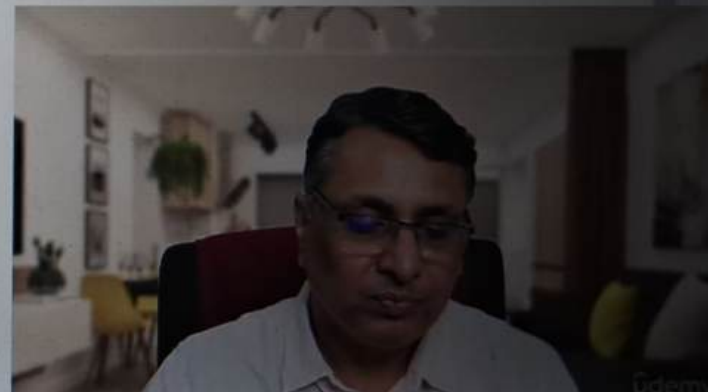
```
def waitForMicroBatch(self, sleep=10):
    import time
    print(f"\tWaiting for {sleep} seconds...", end='')
    time.sleep(sleep)
    print("Done.")
```

```
def assertKafka(self, start_time, expected_count):
    pd = KafkaProducer()
    print(f"\tStarting validation...", end='')
    actual_count = ( spark.read
                    .format("kafka")
                    .option("kafka.bootstrap.servers", pd.BootstrapServer)
                    .option("kafka.security.protocol", "SASL_SSL")
                    .option("kafka.sasl.mechanism", "PLAIN")
                    .option("kafka.sasl.jaas.config", f"{{pd.JAAS_MODULE}} required username='{{pd.ClusterApiKey}}' password='{{pd.ClusterApiSecret}}';")
                    .option("subscribe", "invoices")
                    .option("startingTimestamp", start_time)
                    .option("startingOffsetsByTimestampStrategy", "latest")
                    .load()
                    .count()

    assert expected_count == actual_count, f"Test failed! actual topic count is {actual_count}"
    print("Done")
```

Shift+Enter to run

Shift+Ctrl+Enter to run selected text



```
        .option("startingOffsetsByTimestampStrategy", "latest")
        .load()
        .count()
    )

    assert expected_count == actual_count, f"Test failed! actual topic count is {actual_count}"
    print("Done")

def runTests(self):
    import time
    start_time = round(time.time() * 1000)

    self.cleanTests()
    kp = KafkaProducer()
    kpStream = kp.process("StoreID == 'STR7188'")

    print("Testing first iteration...")
    self.ingestData(1)
    self.waitForMicroBatch()
    self.assertKafka(start_time, 53)
    print("Validation passed.\n")

    print("Testing second iteration...")
    self.ingestData(2)
    self.waitForMicroBatch()
    self.assertKafka(start_time, 53+11)
    print("Validation passed.\n")

    print("Testing third iteration...")
    self.ingestData(3)
    self.waitForMicroBatch()
    self.assertKafka(start_time, 53+11+25)
    print("Validation passed.\n")

    kpStream.stop()
```



15-kafka-producer-test-suite Python ▾

File Edit View Run Help [Last edit was now](#) [Provide feedback](#)

```
self.waitForMicrobatch()  
self.assertKafka(start_time, 53+11+25)  
print("Validation passed.\n")  
  
kpStream.stop()
```

• Uploading command

Cmd 3

```
kpTS = KafkaProducerTestSuite()  
kpTS.runTests()
```

• Uploading command

Shift+Enter to run

Shift+Ctrl+Enter to run selected text